

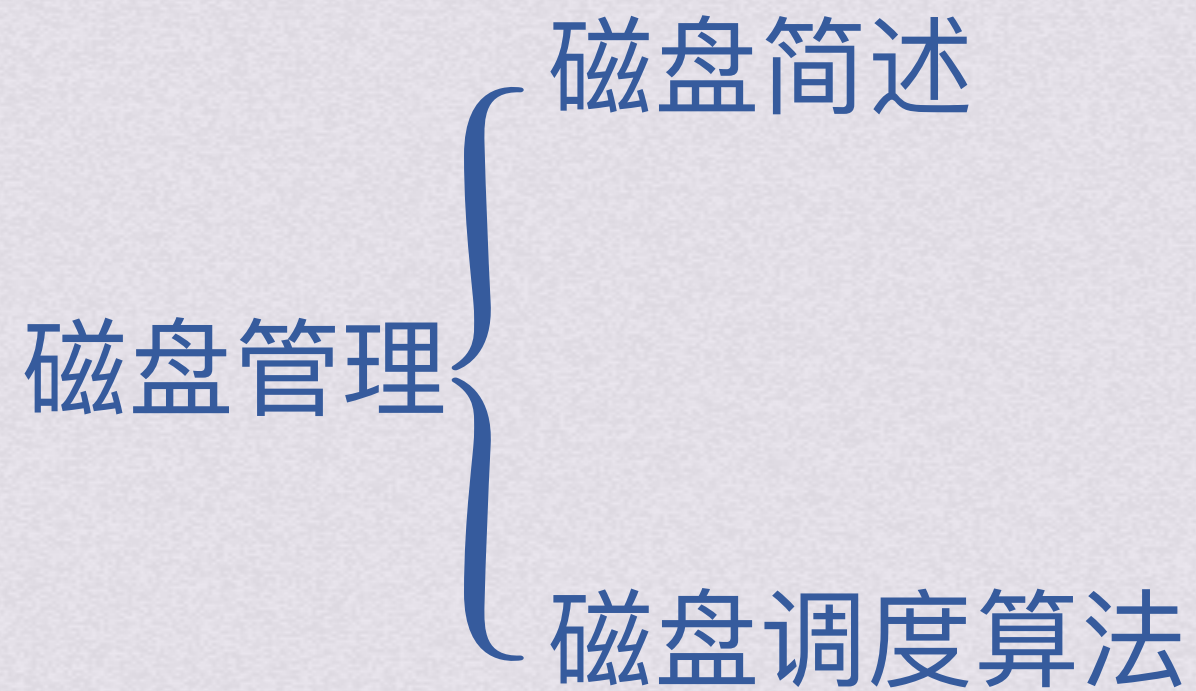
bok25

**基
礎**

課、操作系統



磁盘管理





磁盘简述

*注：本节请结合计组3-6外存一起学习，购买OS单门的同学，课程文件会放在补充资料中

磁盘初始化

新的磁盘是一个磁性记录材料的空白盘，在磁盘可以存储数据之前，必须先将它分成扇区，以便磁盘控制器进行读写操作，这个过程称为**低级格式化(物理格式化)**磁盘工厂在生产时通常就已经将磁盘低级格式化

正式使用磁盘之前，还需要将**磁盘分区**和**逻辑格式化(高级格式化)**

分区是将磁盘分成C盘、D盘等

逻辑格式化是将初始文件系统数据结构存储到磁盘上，这些数据结构包括空闲空间、已分配空间

扇区单位太小，有一些操作系统将多个相邻扇区组合成**簇**来作为基本单位分配给文件



未逻辑格式化的磁盘不会显示在“此电脑”中





磁盘简述

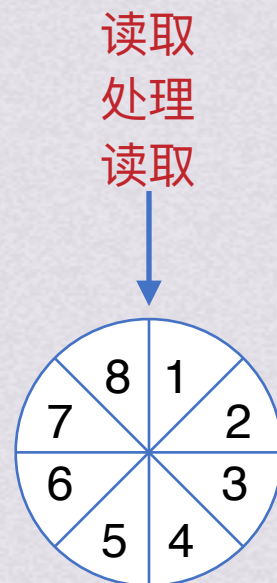
*注：本节请结合计组3-6外存一起学习，购买OS单门的同学，课程文件会放在补充资料中

坏块

磁盘中会有扇区发生损坏，被称为坏块，简单磁盘的坏块可以通过一些命令在FAT表上屏蔽；复杂一些的磁盘控制器会自行维护坏块列表并且不断更新，并且会有一些备用的块用来逻辑上替换坏块，称为**扇区备用**

减少磁盘延迟时间的方法

磁盘是连续自转的设备，读入一个扇区以后需要经过短暂的处理才能开始读下一个扇区



磁盘简述

*注：本节请结合计组3-6外存一起学习，购买OS单门的同学，课程文件会放在补充资料中

坏块

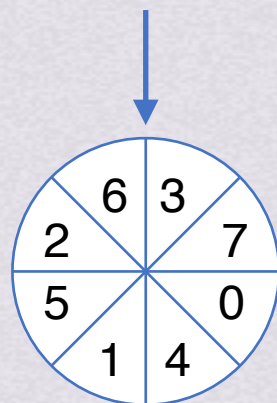
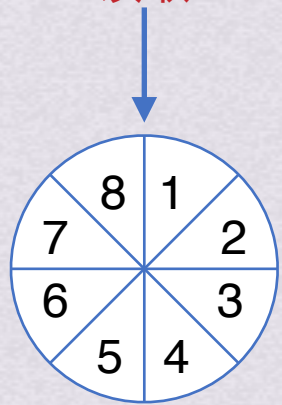
磁盘中会有扇区发生损坏，被称为坏块，简单磁盘的坏块可以通过一些命令在FAT表上屏蔽；复杂一些的磁盘控制器会自行维护坏块列表并且不断更新，并且会有一些备用的块用来逻辑上替换坏块，称为**扇区备用**

减少磁盘延迟时间的方法

磁盘是连续自转的设备，读入一个扇区以后需要经过短暂的处理才能开始读下一个扇区

读取
处理
读取

可以看出，如果顺序读取1,2扇区，那么读完1在处理1的时候旋转到了2，此时因为在处理已读的1的数据而无法处理2



可以对一个盘面的扇区进行交替编号，这样可以减少读入多个连续块时所用时间



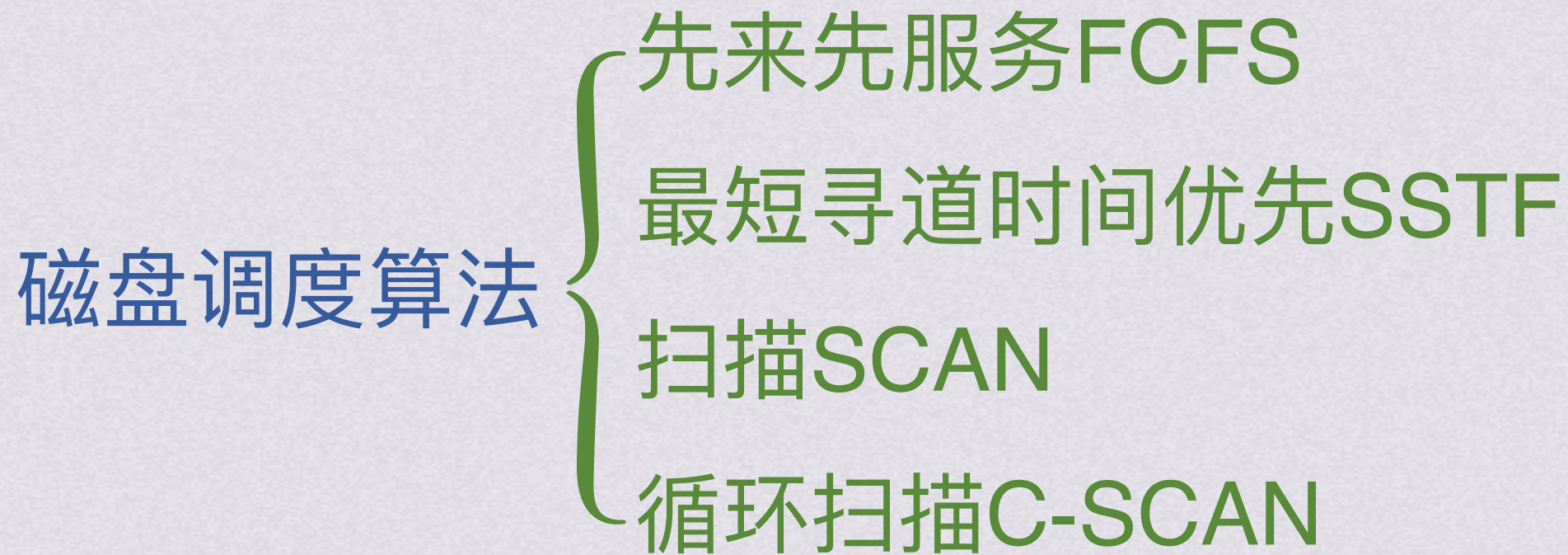
磁盘简述

*注：本节请结合计组3-6外存一起学习，购买OS单门的同学，课程文件会放在补充资料中

固态硬盘的磨损均衡

动态磨损均衡：写入数据的时候会自动选择较新的闪存块写入

静态磨损均衡：ssd会自动检测进行数据分配，让老的闪存块承担无需写数据的存储任务，让较新的闪存块腾出空间，读写操作在较新的闪存快中进行



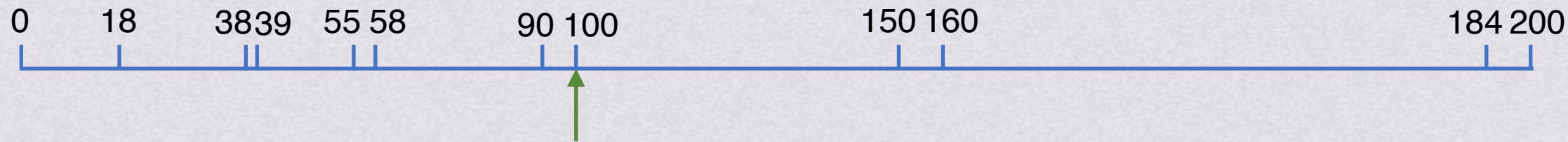


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



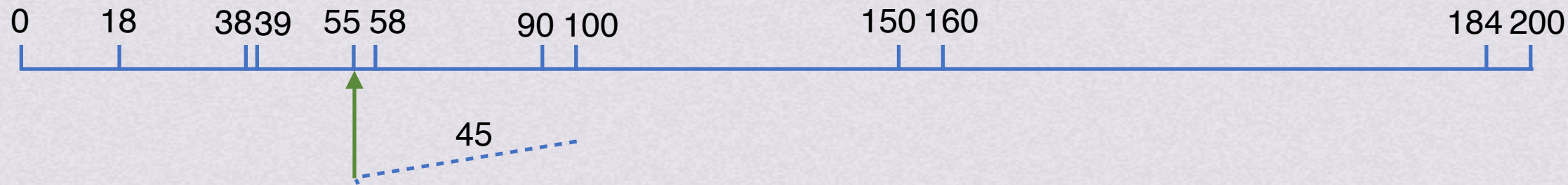


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



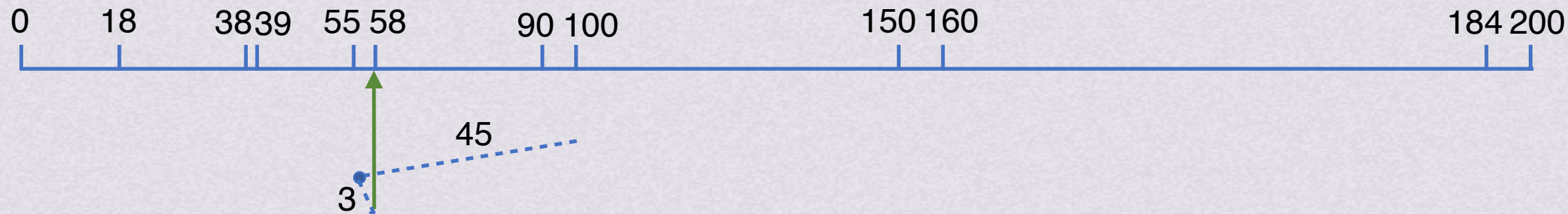


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



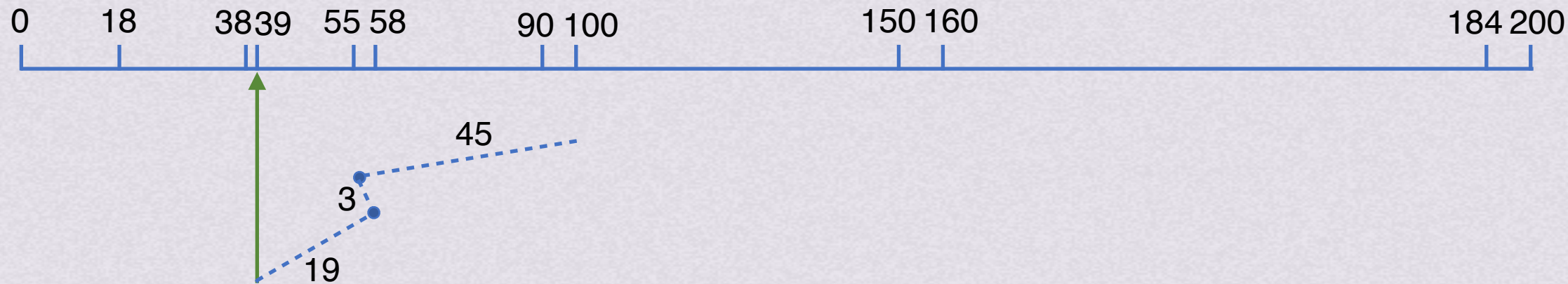


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



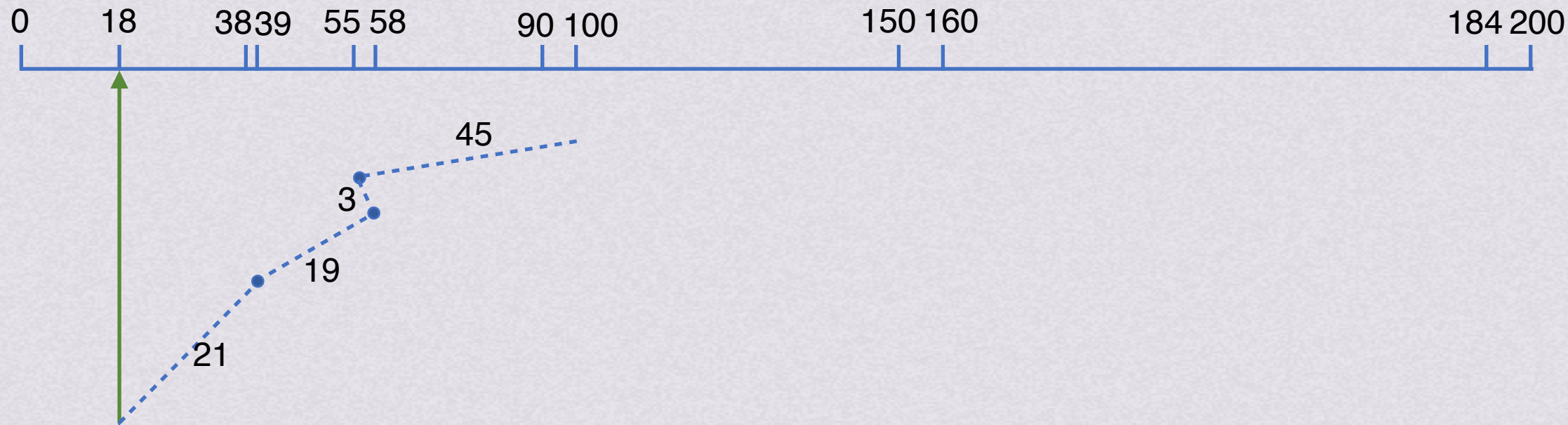


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



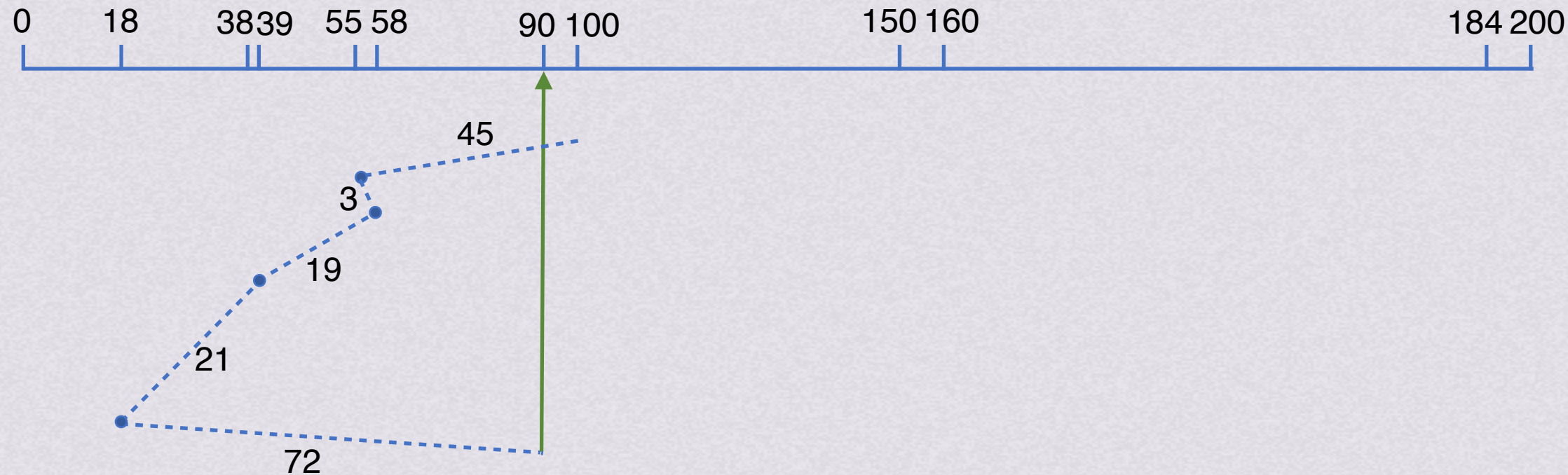


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



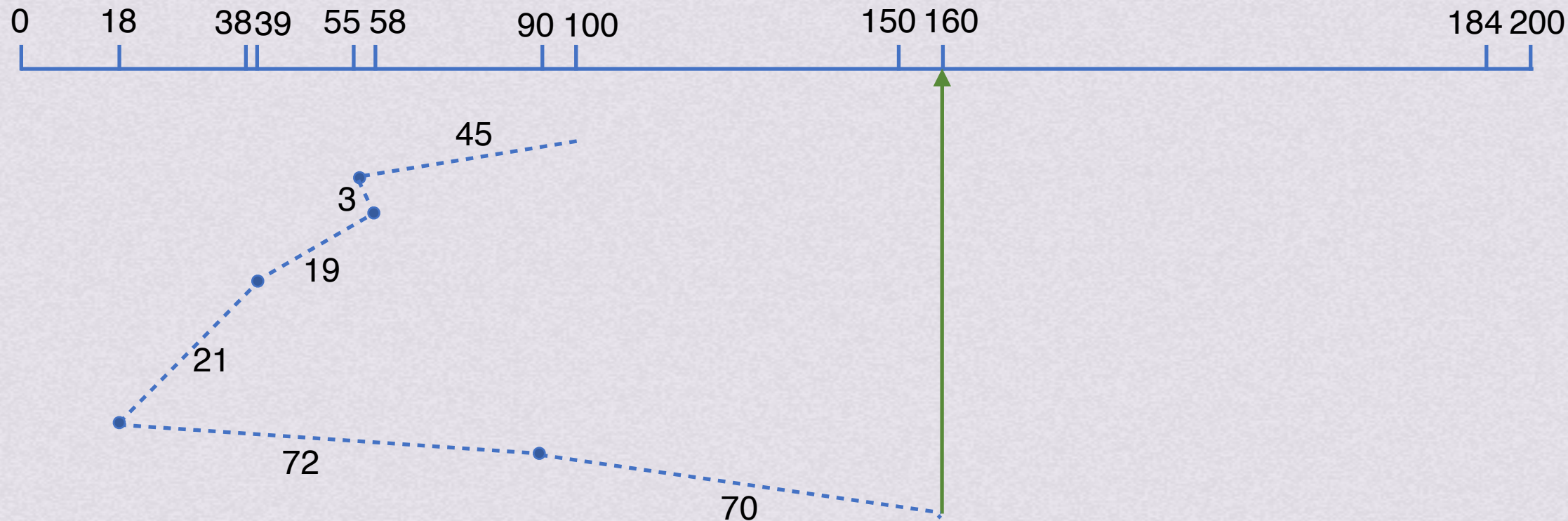


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



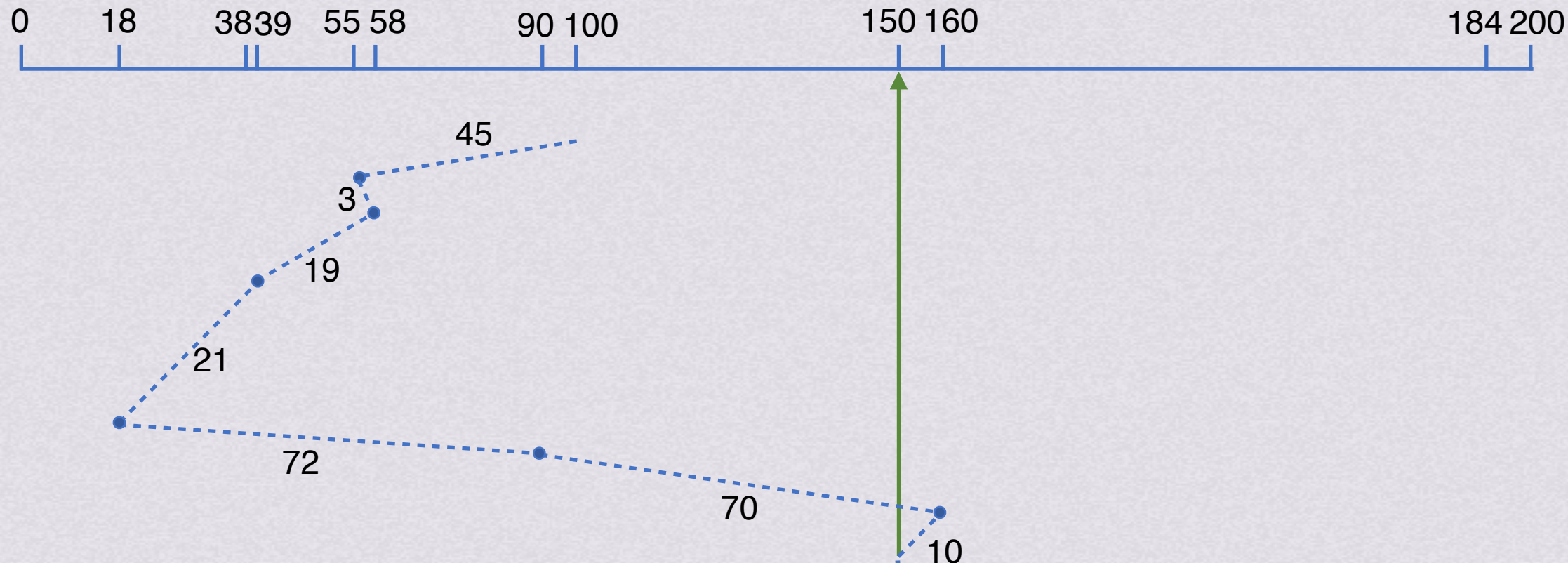


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



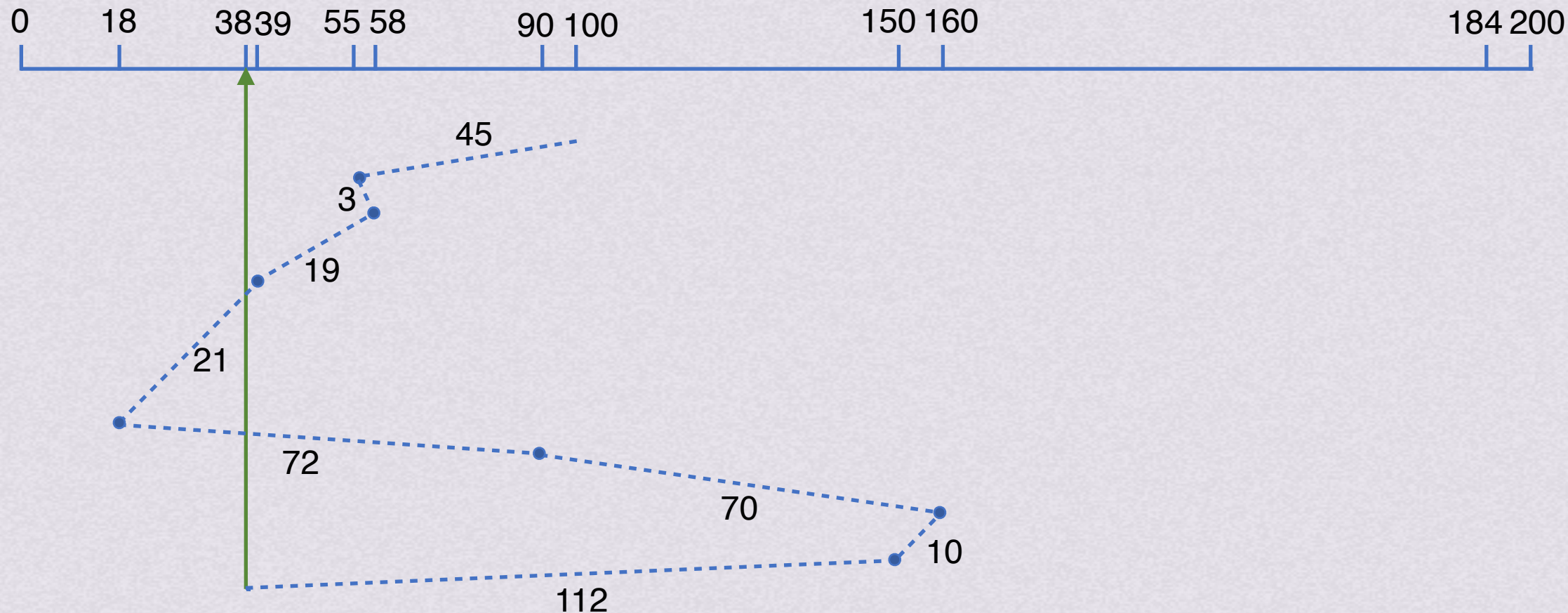


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



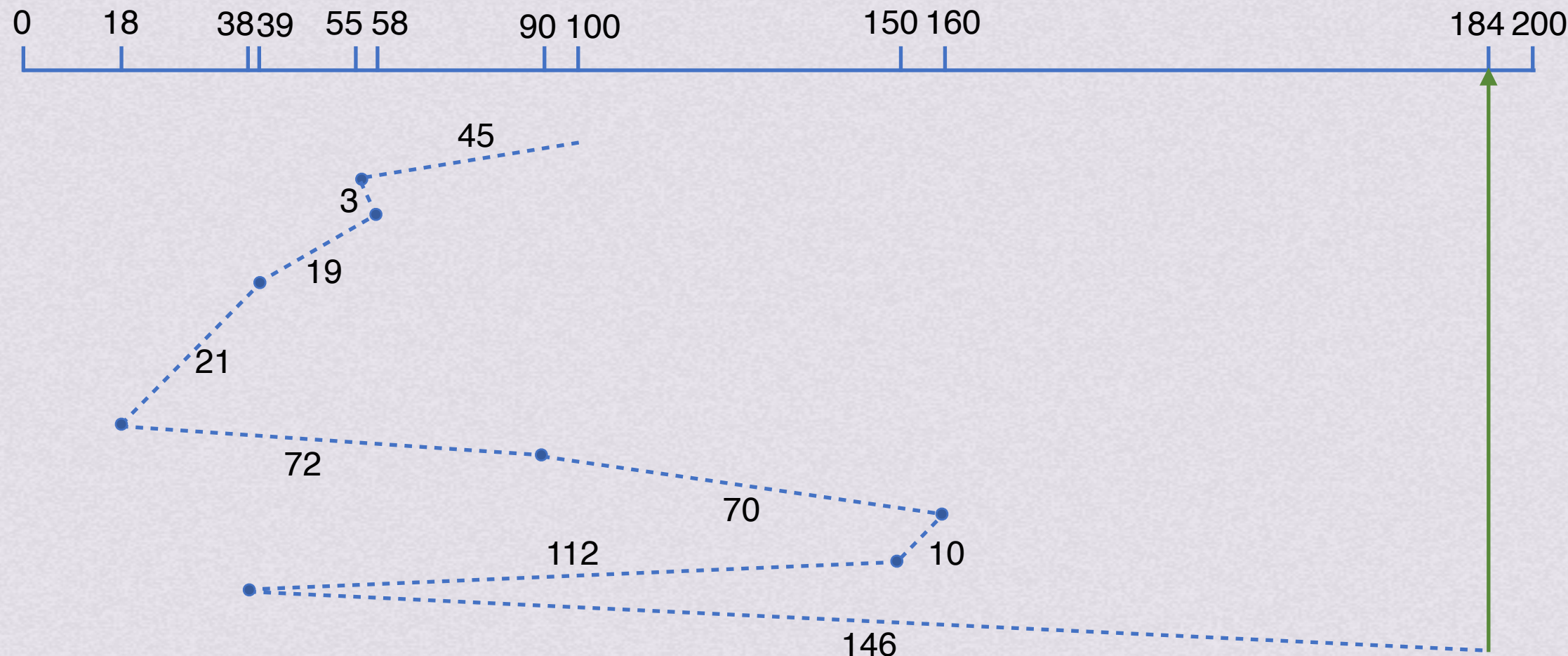


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



磁盘简述

最短寻道时间优先SSTF
扫描SCAN
循环扫描C-SCAN

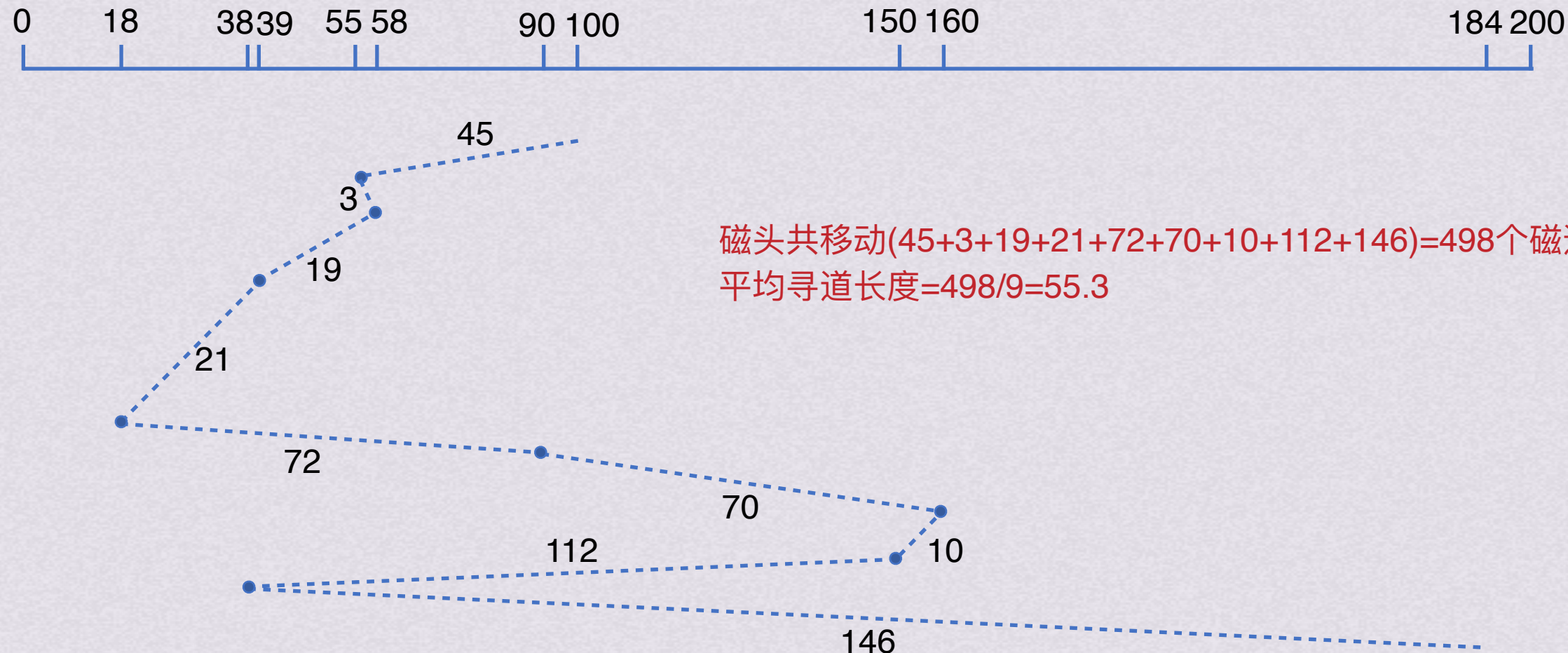


磁盘调度算法

先来先服务FCFS

根据访问磁盘的先后顺序进行调度，是一种最简单的调度算法

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用FCFS算法磁头运动过程如下



磁盘简述

- 最短寻道时间优先SSTF
- 扫描SCAN
- 循环扫描C-SCAN



磁盘调度算法

先来先服务FCFS



磁盘调度算法

最短寻道时间优先SSTF

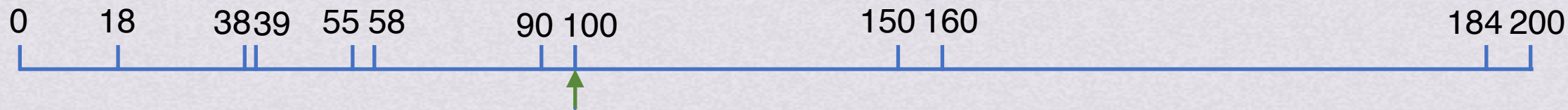


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



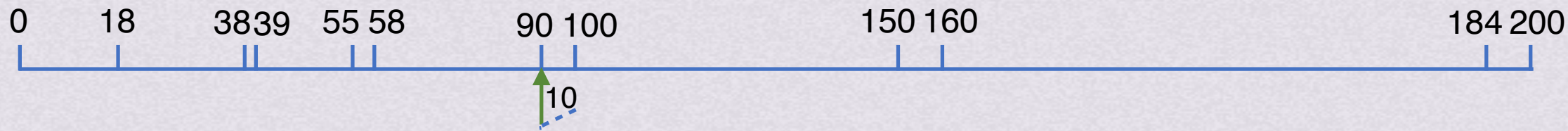


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下





磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



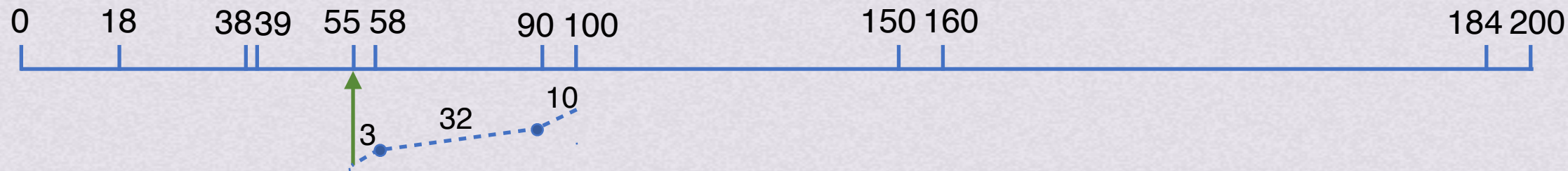


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下





磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



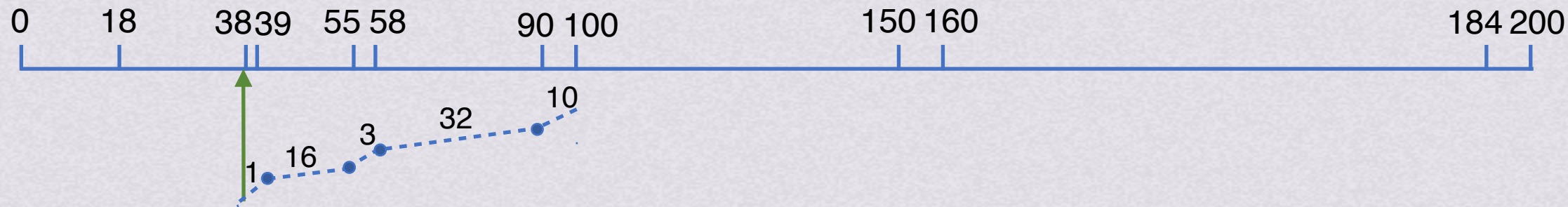


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



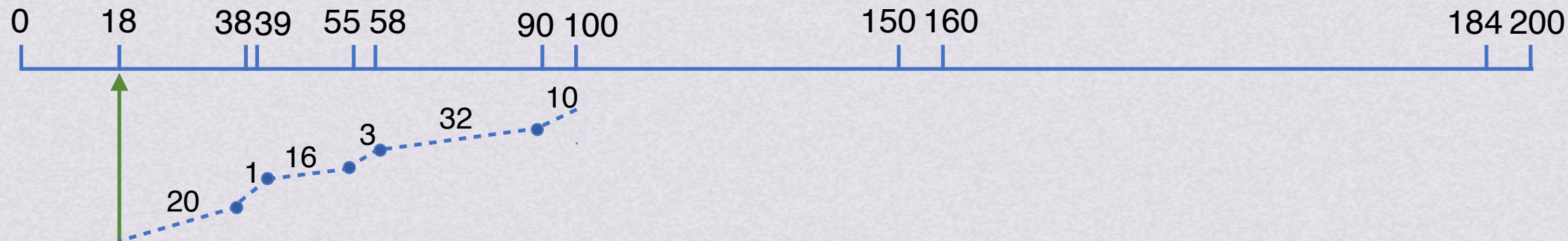


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



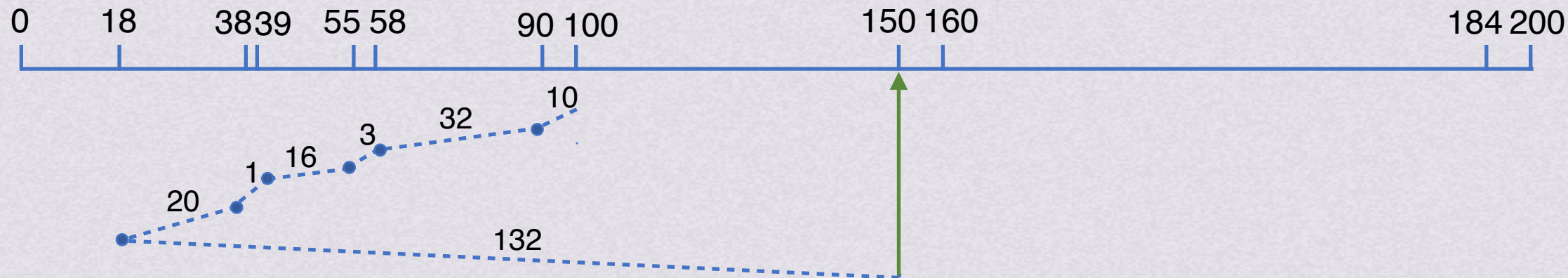


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



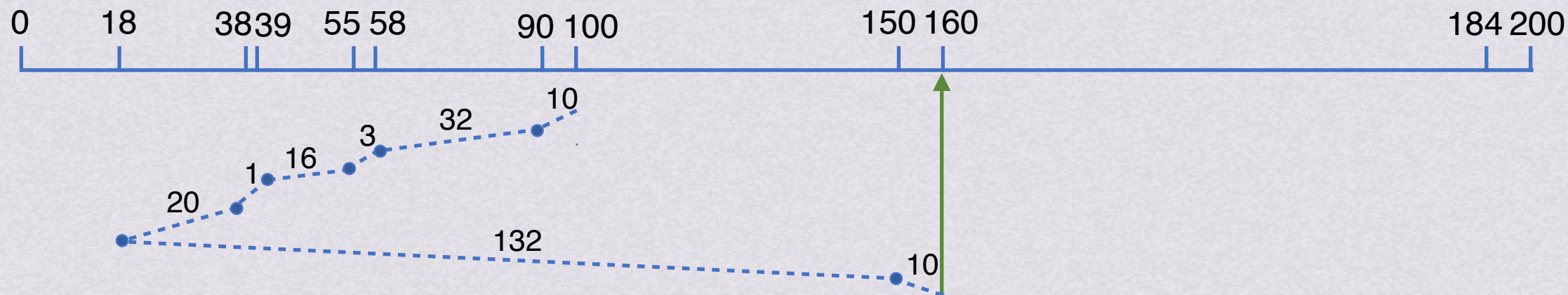


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



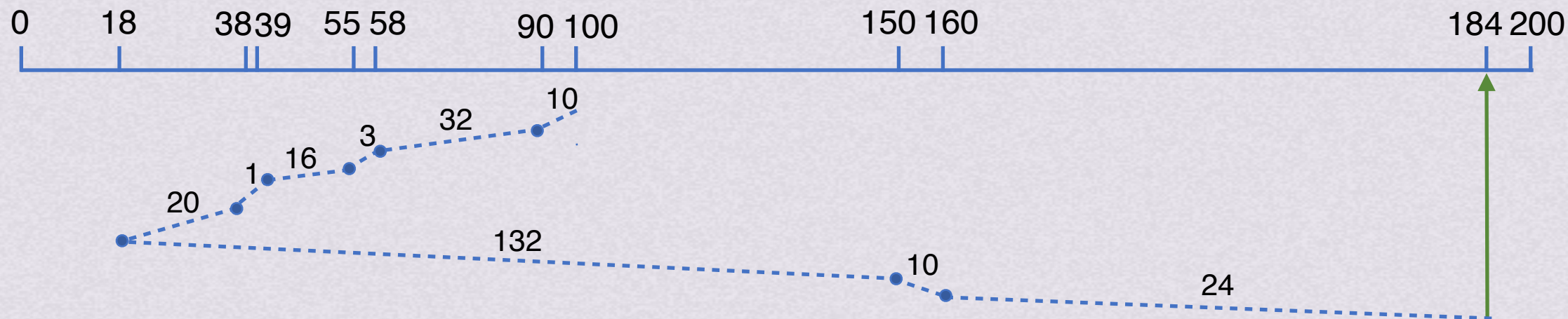


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



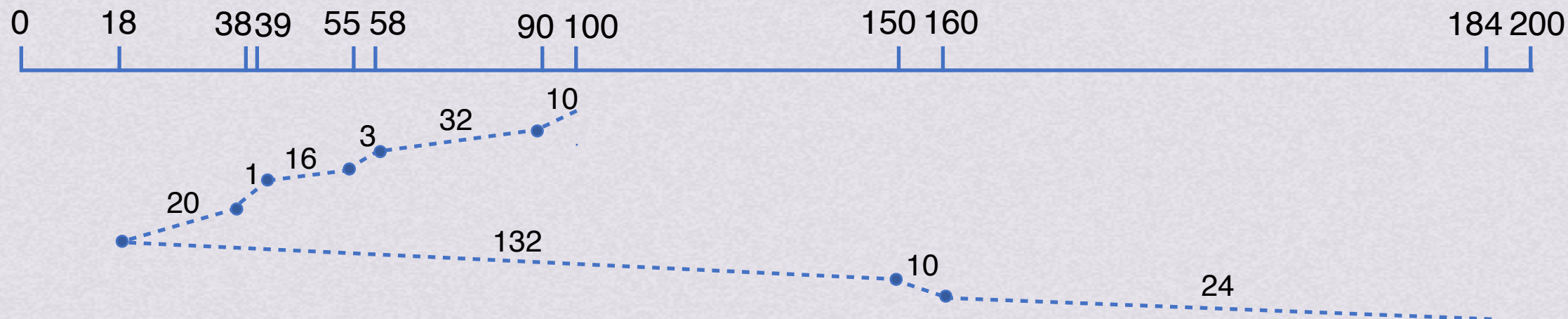


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



磁头共移动 $(10+32+3+16+1+20+132+10+24)=248$ 个磁道

平均寻道长度 $=248/9=27.5$

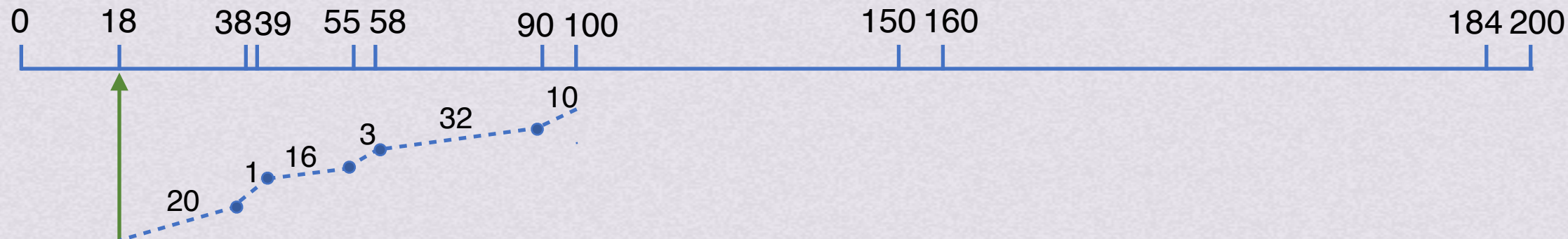


磁盘调度算法

最短寻道时间优先SSTF

每次选择调度与当前磁头最近的磁道，让每次寻道时间最短，这种算法可能产生饥饿现象

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SSTF算法磁头运动过程如下



饥饿现象：假设此时一直有18附近的磁道访问申请，那距离远的磁道请求将一直得不到满足
(如请求顺序中不断加入17,19,17,19.....)



磁盘调度算法

最短寻道时间优先SSTF



磁盘调度算法

扫描SCAN

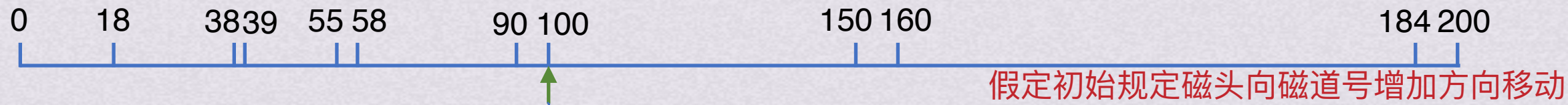


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



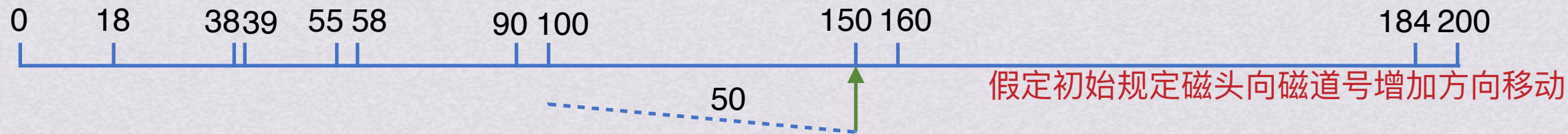


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



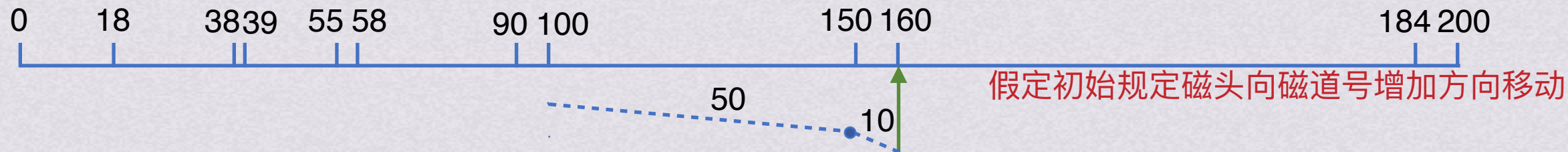


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



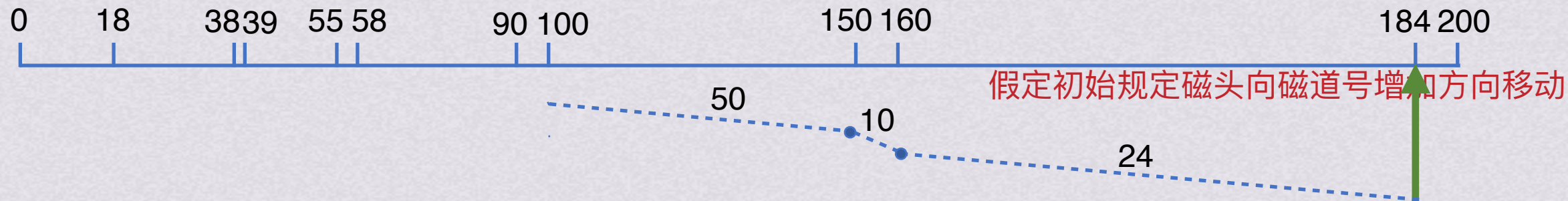


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



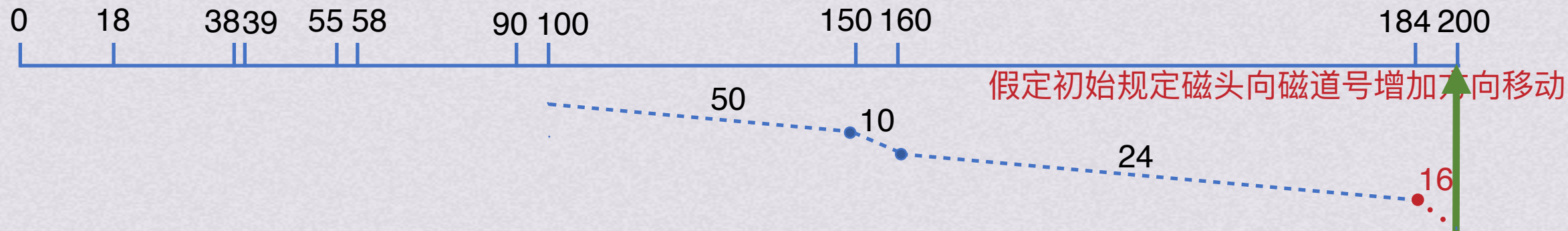


磁盘调度算法

扫描SCAN

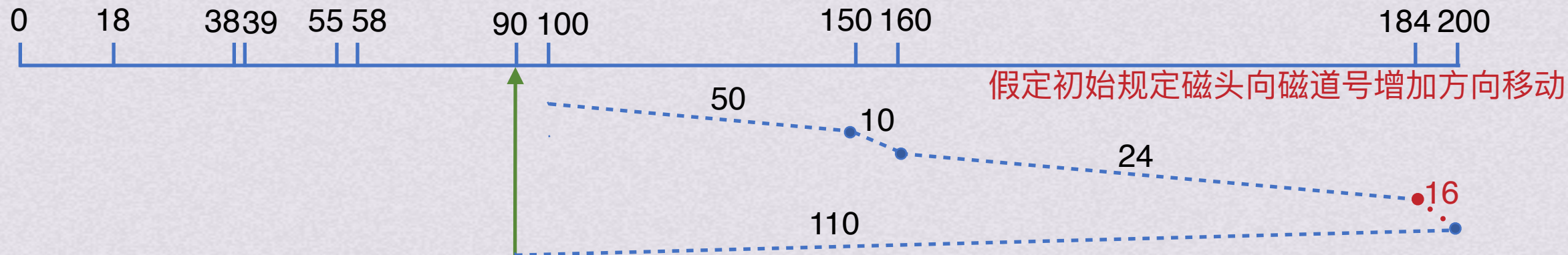
在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



扫描SCAN

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100, 采用SCAN算法磁头运动过程如下



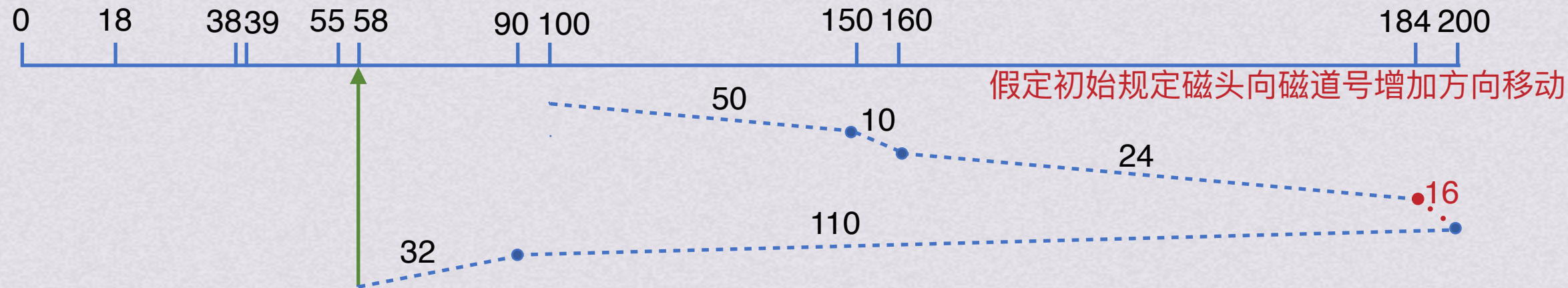


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



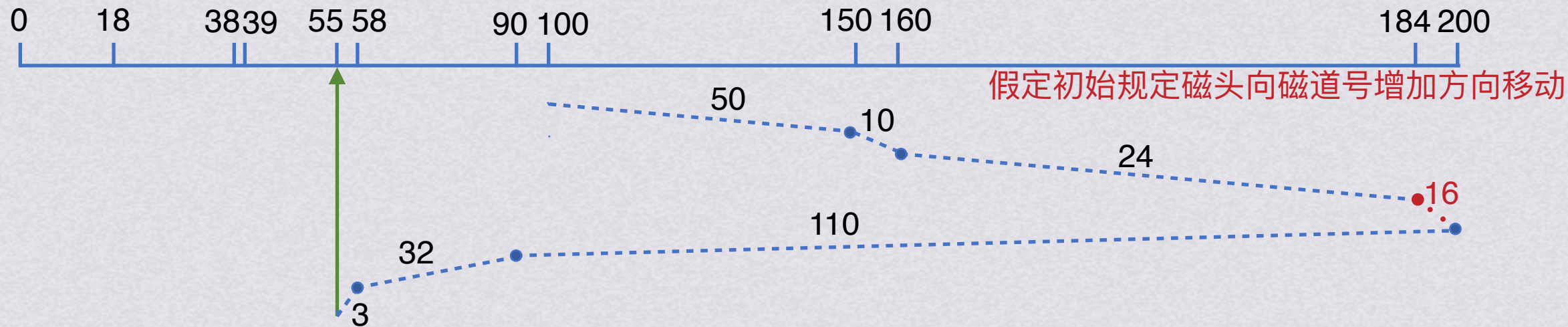


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



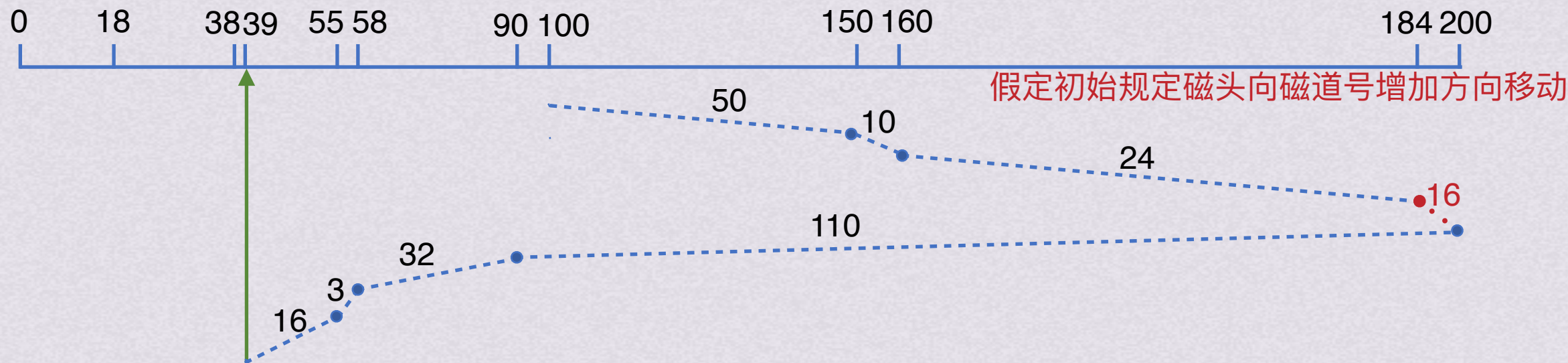


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



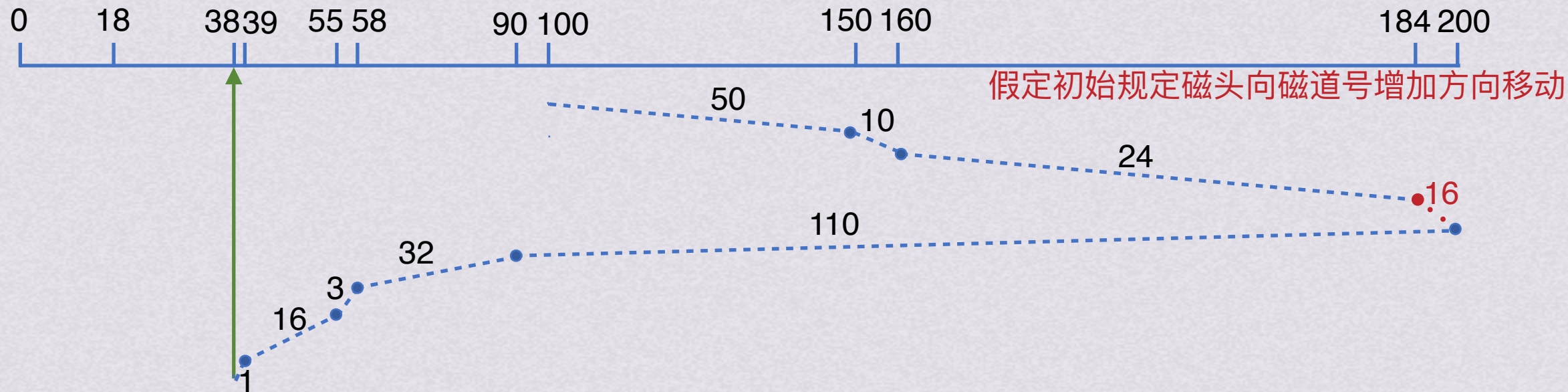


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



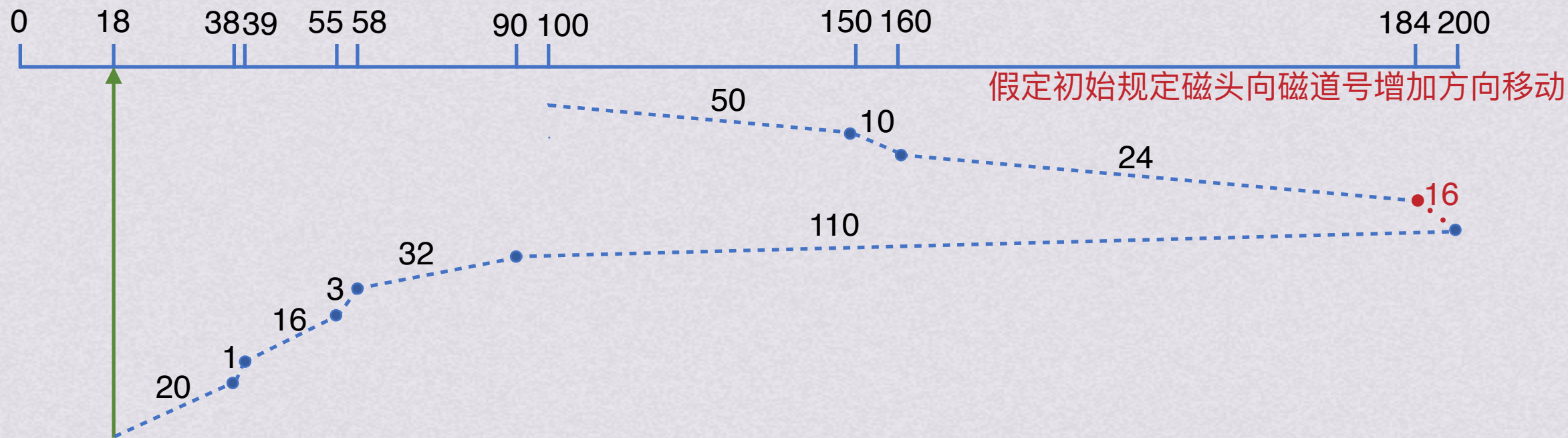


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



磁头共移动 $(50+10+24+16+110+32+3+16+1+20)=282$ 个磁道
平均寻道长度 $=282/9=31.33$

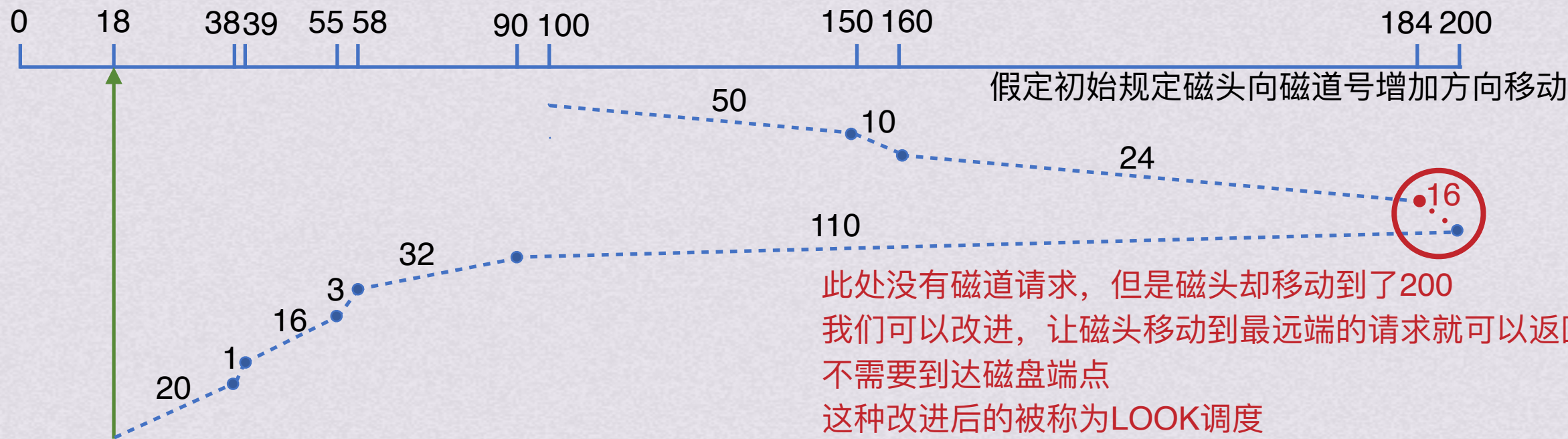


磁盘调度算法

扫描SCAN

在SSTF的基础上规定磁头移动方向，只有碰到最内外侧端点才允许反向移动，来防止产生饥饿

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用SCAN算法磁头运动过程如下



磁头共移动 $(50+10+24+16+110+32+3+16+1+20)=282$ 个磁道

平均寻道长度 $=282/9=31.33$



磁盘调度算法

扫描SCAN



磁盘调度算法

循环扫描C-SCAN



磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



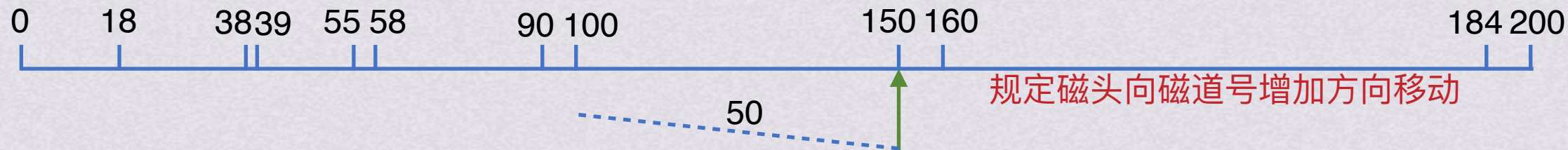


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



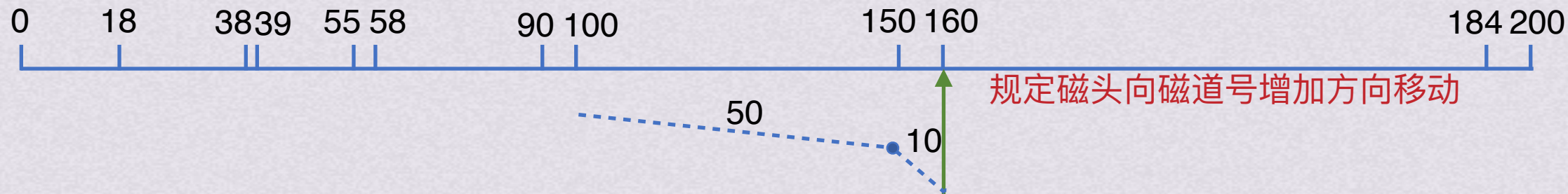


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



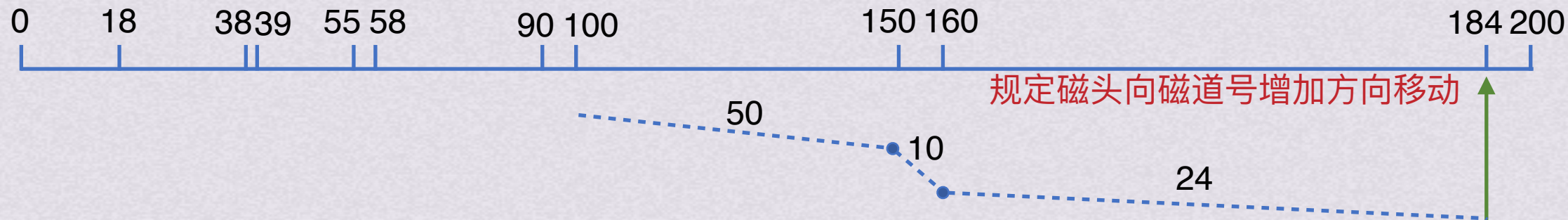


磁盘调度算法

循环扫描C-SCAN

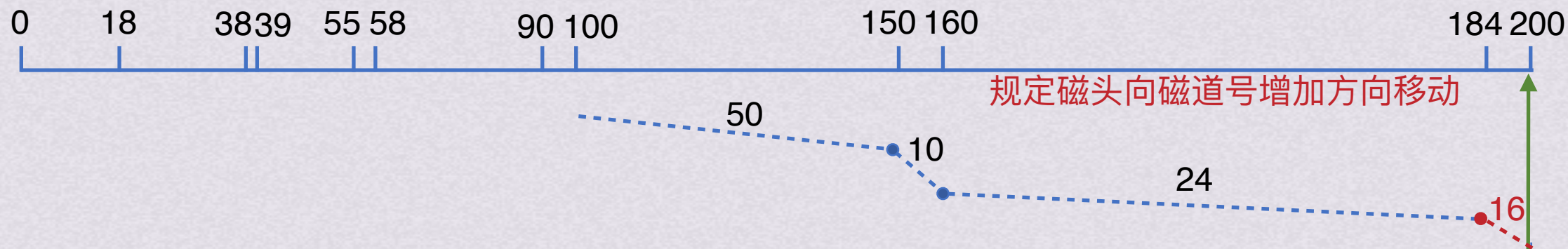
在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



循环扫描C-SCAN

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100, 采用C-SCAN算法磁头运动过程如下



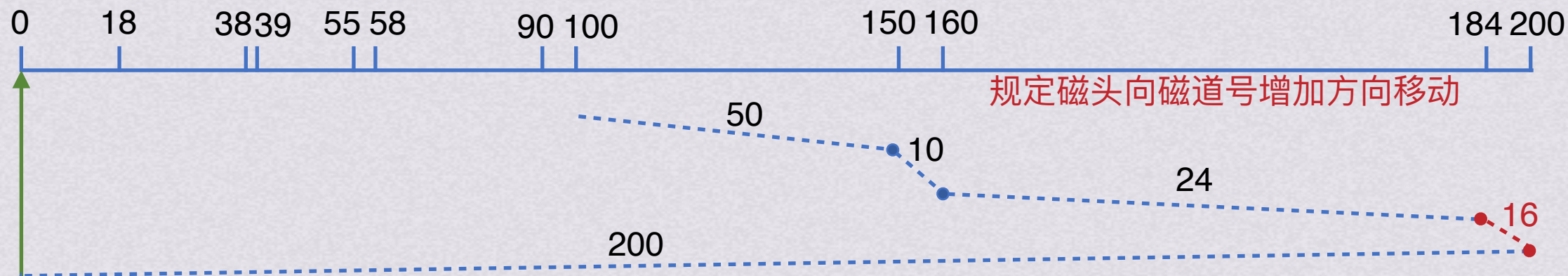


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



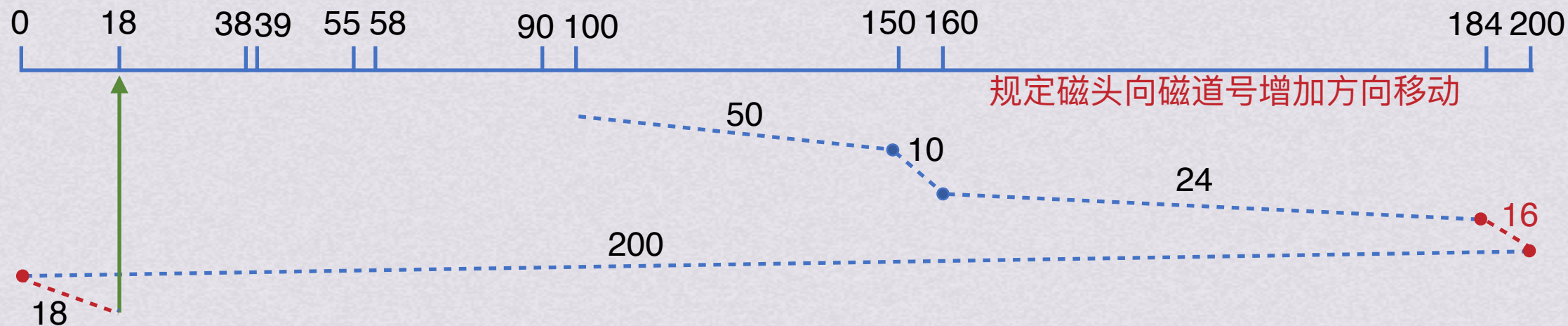


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



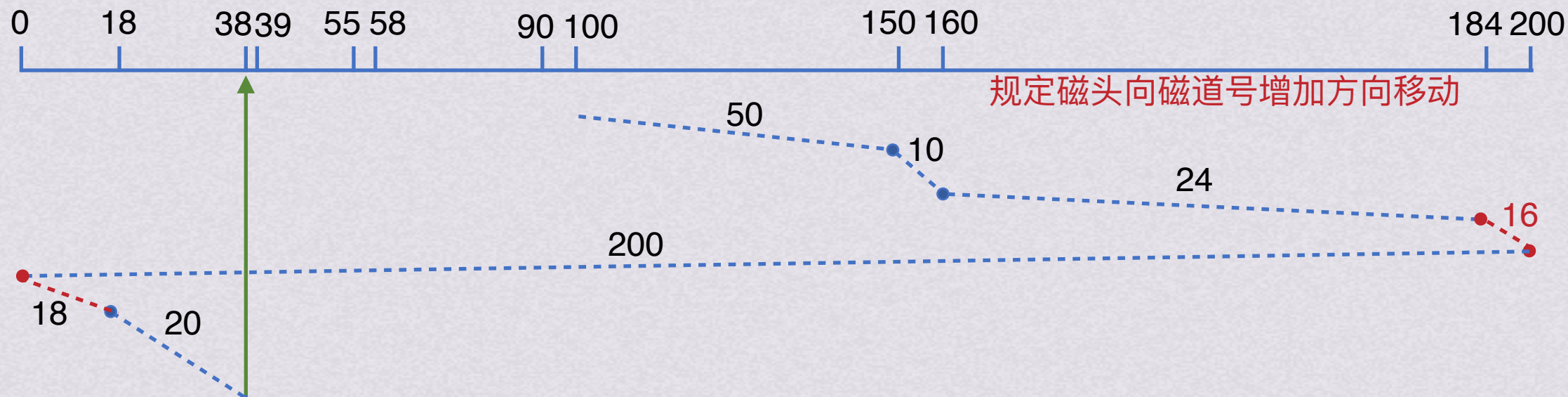


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



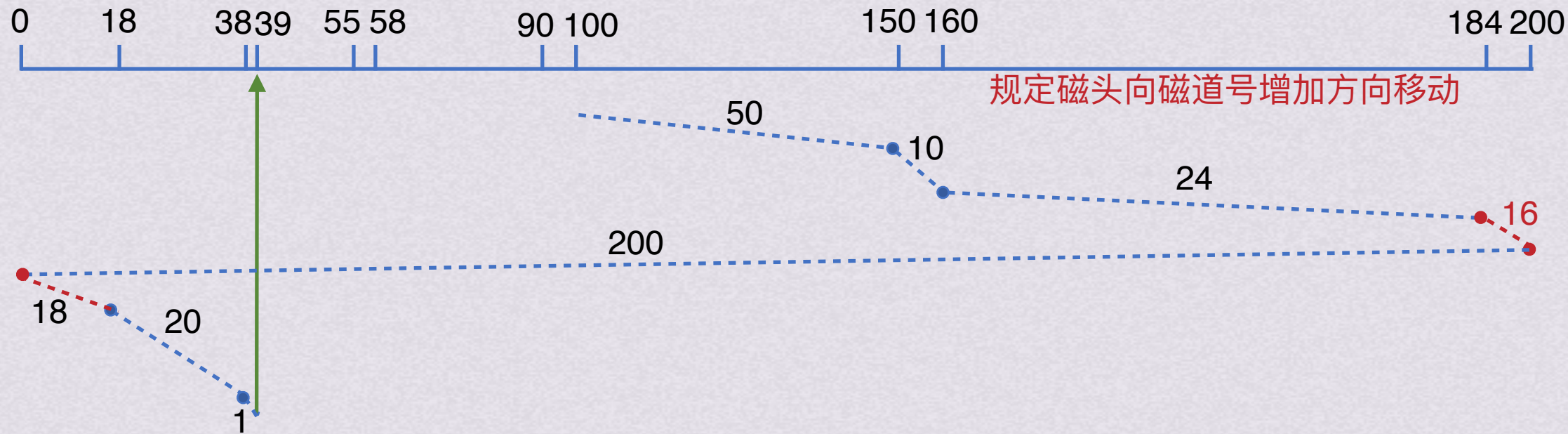


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



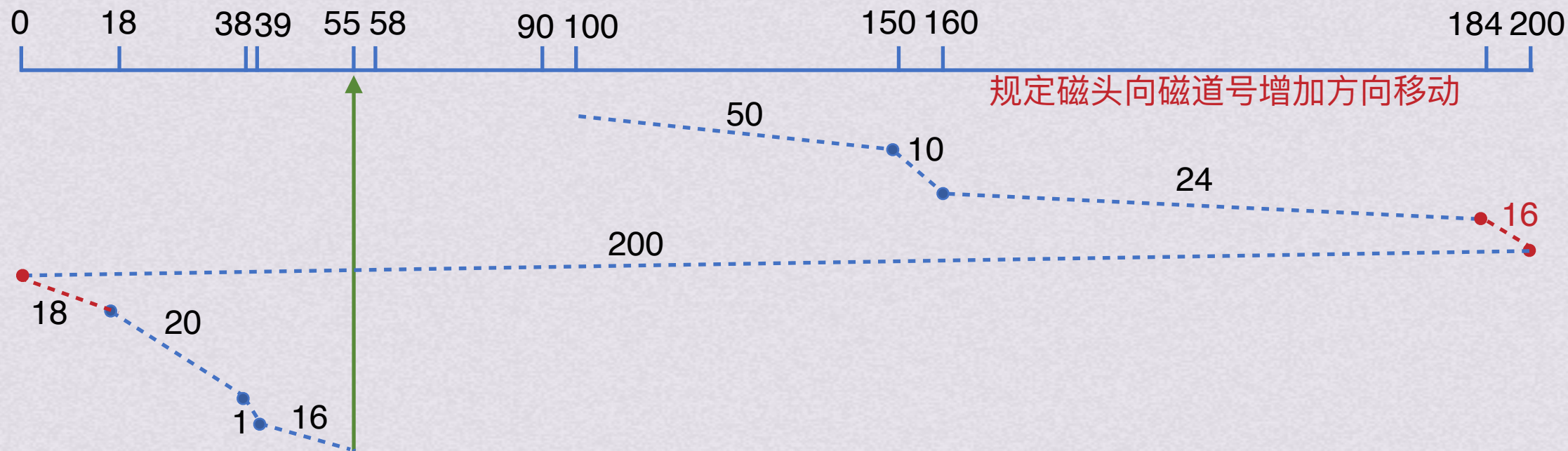


磁盘调度算法

循环扫描C-SCAN

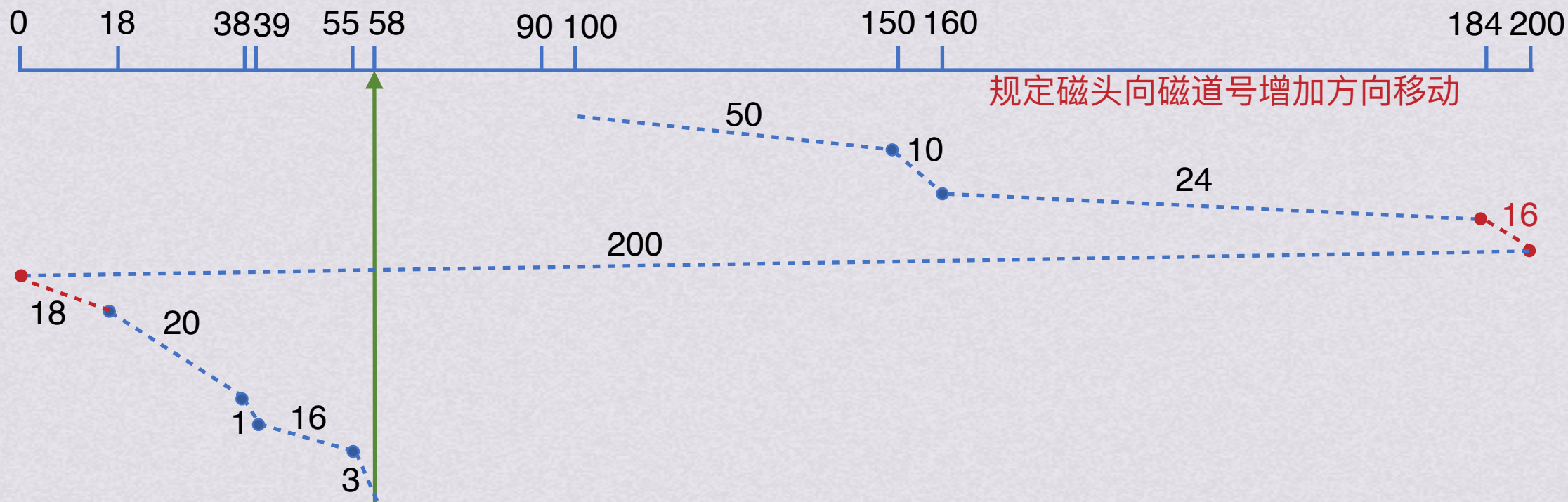
在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下



循环扫描C-SCAN

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100, 采用C-SCAN算法磁头运动过程如下



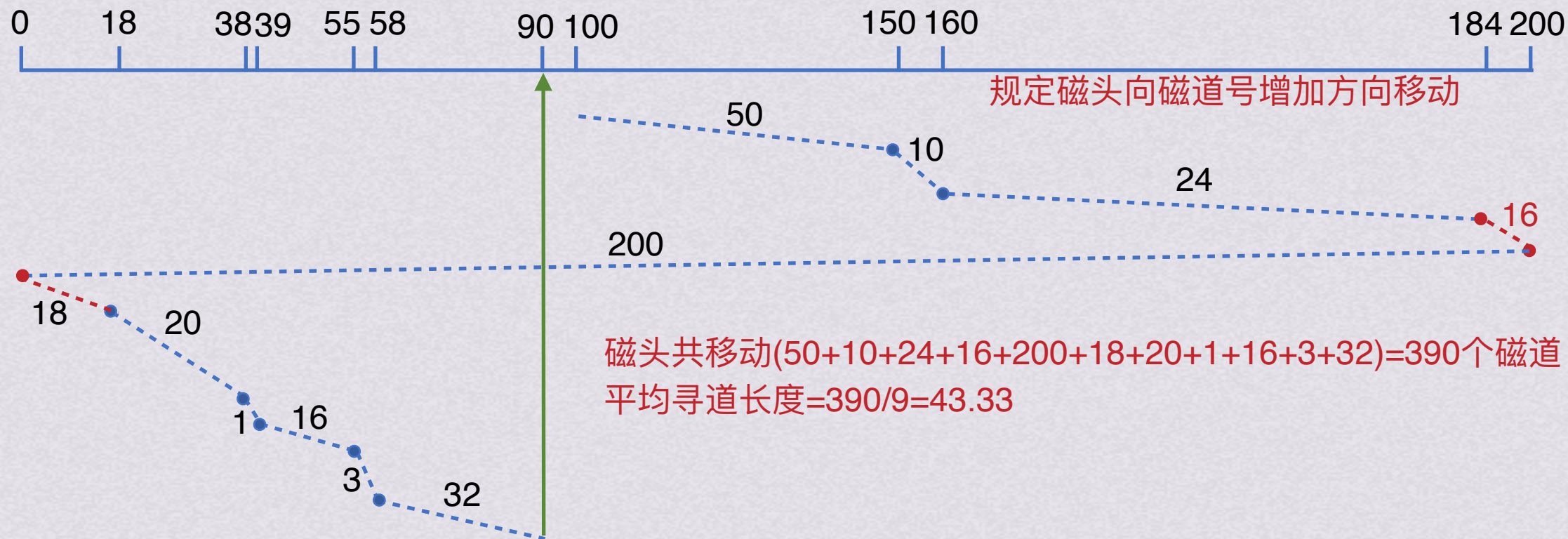


磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下





磁盘调度算法

循环扫描C-SCAN

在SCAN算法的基础上，规定磁头只能单向移动，返回的时候直接返回起始端不服务任何请求

如磁盘请求队列中请求顺序为55,58,39,18,90,160,150,38,184,磁头初始位置100，采用C-SCAN算法磁头运动过程如下

